

ENTERPRISE ARCHITECTURE BLUEPRINT

Cloud Accounting & Business Management Platform

(Codename "Ledger" - Zoho Books-class product)

VOLUME 2 OF 9

APPLICATION ARCHITECTURE

Bounded Contexts - C4 Model - Module Dependencies - UML - Sequence Diagrams

Field	Value
Document Status	Draft - v0.1 - Phase 2 of 9
Prepared For	caldim
Prepared By	Claude (Enterprise Architecture Assistant)
Date	13 July 2026
Classification	Internal - Pre-Development Planning
Upstream Dependency	Vol. 1 Business Architecture (capability IDs BC-01...BC-21, personas PER-01...PER-12, value streams VS-01...VS-06)
Downstream Volumes	Vol. 3 Technical Architecture; Vol. 4 Data Architecture; Vol. 5 Integration Architecture; Vol. 6 Security Architecture; Vol. 7 Infrastructure & Deployment; Vol. 8 Testing & QA; Vol. 9 UI/UX & API Design

Table of Contents

1. Introduction & Scope

1.1 Purpose

This volume translates the 21 business capability domains (BC-01 through BC-21) and six value streams (VS-01 through VS-06) defined in Volume 1 into an application architecture: bounded contexts, service/module boundaries, the C4 model (context, container, and selected component diagrams), a module dependency graph, core domain UML, and the key cross-service sequence diagrams. Every bounded context defined here is traceable to one or more BC-xx capability IDs (Section 3.2); every downstream volume (data model in Vol. 4, API contracts in Vol. 9, security boundaries in Vol. 6) is expected to align to the same boundaries.

1.2 What This Volume Does Not Cover

- Concrete technology stack, language, and framework choices - that is Volume 3 (Technical Architecture).
- Physical database schema and ER diagrams - that is Volume 4 (Data Architecture), though this volume fixes the data ownership boundaries Volume 4 must respect.
- Detailed REST/GraphQL endpoint contracts - that is Volume 9 (API Design Document); this volume only sketches the API surface at the bounded-context level (Section 9).
- Deployment topology, scaling, and infrastructure sizing - that is Volume 7.

2. Architectural Style Decision

2.1 Decision Record: Modular Monolith First, Service-Ready Boundaries

Field	Detail
Status	Accepted
Context	Vol. 1 requires independent testability/deployability in the medium term (Principle P2) and strict multi-tenancy (P1), but the team is starting from zero with an MVP scope (Vol. 1 Section 6). A pure microservices start adds operational overhead (service mesh, distributed transactions, N deployment pipelines) before there is traffic or team headcount to justify it.
Decision	Build a modular monolith for Release 1 (MVP), where each bounded context (Section 3) is a separate module with an enforced internal API boundary (no cross-module direct database access), a private schema/namespace (Vol. 4), and its own event-publishing contract. Deploy as one process initially. Extraction to an independently deployable service is a deployment change, not a re-architecture, because the boundaries are already correct.
Consequences (positive)	Faster MVP delivery; single deployment pipeline at first; easier local development and debugging; no distributed-transaction complexity for Release 1 cross-context flows (Section 8).
Consequences (negative)	Requires discipline to prevent modules from reaching into each other's data (enforced via Vol. 3 coding standards and Vol. 4 schema-per-module); a noisy-neighbor module can still affect the whole process's resource usage until extracted.
Extraction Trigger Criteria	A module is split out into its own deployable service when any of: (a) it needs independent scaling (e.g. Reporting under heavy query load), (b) it needs an independent release cadence (e.g. AI Assistant iterating faster than core Ledger), or (c) it becomes a reliability risk to co-locate (e.g. long-running Payroll batch jobs).

2.2 Communication Style Between Bounded Contexts

Interaction Type	Mechanism	Example
Command (synchronous, needs immediate result)	In-process module API call (Release 1) -> internal RPC once extracted	Receivables asks Inventory "is this SKU in stock" before confirming a sales order
Fact notification (other contexts may react, sender does not need a response)	Domain event on the internal event bus (Section 7)	Ledger publishes JournalPosted; Reporting and Analytics consume it
Query across context for read models	Context-owned read API, never direct DB read	Reporting queries Receivables' read API for invoice aging, never reads Receivables' tables directly

3. Domain-Driven Design: Bounded Contexts

3.1 Context Consolidation Rationale

The 21 capability domains from Vol. 1 are consolidated into 16 bounded contexts. Consolidation follows standard DDD guidance: capabilities that share a ubiquitous language, change for the same business reasons, and are usually modified together are merged (e.g. Expense Management folds into Payables since both represent money leaving the business); capabilities with genuinely distinct language and change cadence stay separate (e.g. Tax Compliance is distinct from Ledger even though both touch financial postings, because tax rules change on regulatory schedules, not accounting schedules).

3.2 Bounded Context Catalog

Context ID	Bounded Context	BC-xx Mapped	Type
CTX-01	Identity & Tenant	BC-20, BC-21	Foundational (Core)
CTX-02	Ledger	BC-01, BC-08	Core Domain
CTX-03	Receivables	BC-02	Core Domain
CTX-04	Payables	BC-03, BC-04	Core Domain
CTX-05	Banking	BC-05	Core Domain
CTX-06	Payments	BC-06	Supporting
CTX-07	Inventory	BC-07	Core Domain
CTX-08	Projects	BC-09	Supporting
CTX-09	Payroll	BC-10	Supporting (R3)
CTX-10	Tax & Compliance	BC-11, BC-12	Supporting
CTX-11	Reporting & Analytics	BC-13	Supporting (read-only aggregator)
CTX-12	Documents	BC-14	Generic
CTX-13	Collaboration & Portals	BC-15	Supporting
CTX-14	Automation & Customization	BC-16, BC-17	Supporting
CTX-15	AI Assistant	BC-18	Supporting (R2/R3)
CTX-16	Integration Gateway	BC-19	Generic

3.3 Context Map

Diagram: Context map - relationships between bounded contexts

flowchart TB
 Identity[CTX-01 Identity & Tenant]
 Ledger[CTX-02 Ledger]
 Receivables[CTX-03 Receivables]
 Payables[CTX-04 Payables]
 Banking[CTX-05 Banking]
 Payments[CTX-06 Payments]
 Inventory[CTX-07 Inventory]
 Projects[CTX-08 Projects]

Payroll[CTX-09 Payroll]
 Tax[CTX-10 Tax & Compliance]
 Reporting[CTX-11 Reporting & Analytics]
 Documents[CTX-12 Documents]
 Collab[CTX-13 Collaboration & Portals]
 Automation[CTX-14 Automation & Customization]
 AI[CTX-15 AI Assistant]
 Integration[CTX-16 Integration Gateway]

Identity -.shared kernel.-> Ledger
 Identity -.shared kernel.-> Receivables
 Identity -.shared kernel.-> Payables
 Identity -.shared kernel.-> Inventory
 Receivables -->|customer/upstream| Ledger
 Payables -->|customer/upstream| Ledger
 Payroll -->|customer/upstream| Ledger
 Projects -->|customer/upstream| Receivables
 Receivables -->|conformist| Inventory
 Payables -->|conformist| Inventory
 Receivables -->|conformist| Payments
 Payables -->|conformist| Banking
 Receivables -->|conformist| Banking
 Tax -->|open host service| Receivables
 Tax -->|open host service| Payables
 Reporting -->|downstream, read-only| Ledger
 Reporting -->|downstream, read-only| Receivables
 Reporting -->|downstream, read-only| Payables
 Reporting -->|downstream, read-only| Inventory
 Automation -->|anti-corruption layer| Receivables
 Automation -->|anti-corruption layer| Payables
 AI -->|downstream, read-only| Reporting
 Integration -->|anti-corruption layer| Banking
 Integration -->|anti-corruption layer| Payments
 Integration -->|anti-corruption layer| Inventory
 Collab -->|conformist| Receivables
 Collab -->|conformist| Payables
 Documents -.generic subdomain.-> Receivables
 Documents -.generic subdomain.-> Payables

Relationship legend: "shared kernel" = every context depends on Identity & Tenant for auth/tenancy context (cannot function without it). "customer/upstream" = the pointing context is upstream and the arrow target adapts to it (e.g. Receivables posts journal entries in the shape Ledger dictates). "conformist" = the dependent context accepts the upstream context's model as-is rather than translating it. "open host service" = Tax publishes a stable public interface multiple contexts call. "anti-corruption layer" = a translation layer isolates the context from an external or volatile model (third-party bank/payment APIs, or user-defined automation rules).

4. C4 Model - Level 1: System Context

The system context diagram shows the platform as a single box, its human actors (from Vol. 1 Section 4 personas), and the external systems it must integrate with (from Vol. 1 BC-19 and BC-05/06/11).

Diagram: C4 Level 1 - System Context Diagram

```

flowchart TB
    Owner((PER-01 Business Owner))
    Book((PER-02 Bookkeeper))
    ExtAcc((PER-03 External Accountant))
    Sales((PER-04 Sales Clerk))
    Purch((PER-05 AP Clerk))
    Whse((PER-06 Inventory Mgr))
    PM((PER-07 Project Manager))
    Payr((PER-08 Payroll Admin))
    Cust((PER-09 Customer))
    Vend((PER-10 Vendor))
    Admin((PER-11 System Admin))

    subgraph SYS[Ledger Platform - Cloud Accounting & Business Management System]
        BankAPI[Bank Feed Providers - e.g. Plaid/Yodlee-class]
        PayGW[Payment Gateways - Stripe/Razorpay-class]
        TaxSvc[Tax/E-filing Services - e.g. IRS IRIS, GST portals]
        Ecom[E-commerce Channels - Shopify, Amazon, Walmart]
        EmailSvc[Transactional Email/SMS Provider]
        IdP[External Identity Provider - SSO/SAML]
    end

    Owner --> SYS
    Book --> SYS
    ExtAcc --> SYS
    Sales --> SYS
    Purch --> SYS
    Whse --> SYS
    PM --> SYS
    Payr --> SYS
    Admin --> SYS
    Cust -->|customer portal| SYS
    Vend -->|vendor portal| SYS

    SYS <--> BankAPI
    SYS <--> PayGW
    SYS <--> TaxSvc
    SYS <--> Ecom
    SYS --> EmailSvc
    SYS <--> IdP
  
```

5. C4 Model - Level 2: Container Diagram

Containers represent the deployable/runnable units. Consistent with the Section 2 decision, most business-logic containers are modules within a single "Core Application" process at Release 1, while client apps, the API gateway, event bus, and data stores are always separate containers.

Diagram: C4 Level 2 - Container Diagram

```

flowchart TB
    WebApp[Web App - SPA]
    MobileApp[Mobile App]
    DesktopApp[Desktop App - R3]
    PortalApp[Customer/Vendor Portal - SPA]

    Gateway[API Gateway / BFF]

    subgraph Core[Core Application Process - Modular Monolith, Release 1]
        M1[CTX-01 Identity & Tenant Module]
        M2[CTX-02 Ledger Module]
        M3[CTX-03 Receivables Module]
        M4[CTX-04 Payables Module]
        M5[CTX-05 Banking Module]
        M6[CTX-07 Inventory Module]
        M7[CTX-10 Tax Module]
        M8[CTX-11 Reporting Module]
        M9[CTX-14 Automation Module]
    end

    EventBus{{Event Bus / Message Broker}}
    Cache[(Cache - Redis-class)]
    PrimaryDB[(Primary Datastore - per-tenant schema, Vol.4)]
    SearchIdx[(Search Index)]
    FileStore[(Object/File Storage)]

    IntegrationSvc[Integration Gateway Service - CTX-16]
    PaymentsSvc[Payments Service - CTX-06]
    AISvc[AI Assistant Service - CTX-15, R2/R3]
    PayrollSvc[Payroll Service - CTX-09, R3]

    WebApp --> Gateway
    MobileApp --> Gateway
    DesktopApp --> Gateway
    PortalApp --> Gateway

    Gateway --> Core
    Gateway --> IntegrationSvc
    Gateway --> PaymentsSvc
    Gateway --> AISvc

    Core --> PrimaryDB
    Core --> Cache
    Core --> EventBus
    Core --> FileStore
    M8 --> SearchIdx

    EventBus --> IntegrationSvc
    EventBus --> PaymentsSvc
    EventBus --> AISvc
    EventBus --> PayrollSvc
    PayrollSvc --> PrimaryDB
  
```

IntegrationSvc -->|external calls| BankAPIandGateways[External Systems - see C4 L1]

6. C4 Model - Level 3: Component Diagrams

Component-level detail is shown for the two highest-risk modules identified in Vol. 1 Section 8: Receivables (invoice lifecycle) and Ledger (financial source of truth). Other modules follow the same internal layering (API/Controller -> Domain Service -> Repository -> Event Publisher) and are not separately diagrammed here to avoid redundancy; Volume 9 will detail their API surface.

6.1 Receivables Module - Components

Diagram: C4 Level 3 - Receivables Module components

```

flowchart TB
    subgraph Receivables[CTX-03 Receivables Module]
        API[Invoice/Quote/SalesOrder API Layer]
        DomSvc[Receivables Domain Service - invoice lifecycle, revenue recognition rules]
        ApprovalEngine[Sales Approval Rule Evaluator]
        RecurEngine[Recurring Invoice Scheduler]
        Repo[Receivables Repository]
        Publisher[Domain Event Publisher]
    end
    API --> DomSvc
    DomSvc --> ApprovalEngine
    DomSvc --> RecurEngine
    DomSvc --> Repo
    DomSvc --> Publisher
    Repo --> DB[(Receivables Schema)]
    Publisher --> Bus[{Event Bus}]
    DomSvc -.calls.-> InventoryAPI[[Inventory Module API]]
    DomSvc -.calls.-> LedgerAPI[[Ledger Module API]]
    DomSvc -.calls.-> TaxAPI[[Tax Module API]]

```

6.2 Ledger Module - Components

Diagram: C4 Level 3 - Ledger Module components

```

flowchart TB
    subgraph Ledger[CTX-02 Ledger Module]
        API[Journal/Account API Layer]
        Posting[Posting Engine - double-entry validation]
        COA[Chart of Accounts Manager]
        Lock[Period Lock Enforcer]
        FX[Base Currency Revaluation Service]
        Repo[Ledger Repository - append-only journal store]
        Publisher[Domain Event Publisher]
    end
    API --> Posting
    Posting --> COA
    Posting --> Lock
    Posting --> Repo
    FX --> Repo
    Posting --> Publisher
    Publisher --> Bus[{Event Bus}]
    Repo --> DB[(Ledger Schema - append-only)]
    Bus -.consumed by.-> Reporting[[Reporting Module]]

```

7. Module Dependency Diagram

This is the authoritative dependency graph for build order and for enforcing "no upward dependency" rules in Volume 3's coding standards (a lower-layer module must never import from a higher-layer module). Layering is derived directly from Section 3's context map.

Diagram: Module dependency graph (layered)

```

flowchart BT
  subgraph L0[Layer 0 - Foundational]
    Identity[CTX-01 Identity & Tenant]
  end
  subgraph L1[Layer 1 - Core Domain]
    Ledger[CTX-02 Ledger]
  end
  subgraph L2[Layer 2 - Core Business Modules]
    Receivables[CTX-03 Receivables]
    Payables[CTX-04 Payables]
    Inventory[CTX-07 Inventory]
    Banking[CTX-05 Banking]
  end
  subgraph L3[Layer 3 - Supporting Modules]
    Payments[CTX-06 Payments]
    Projects[CTX-08 Projects]
    Tax[CTX-10 Tax & Compliance]
    Documents[CTX-12 Documents]
    Collab[CTX-13 Collaboration & Portals]
  end
  subgraph L4[Layer 4 - Cross-Cutting / Aggregating]
    Reporting[CTX-11 Reporting & Analytics]
    Automation[CTX-14 Automation & Customization]
    AI[CTX-15 AI Assistant]
    Integration[CTX-16 Integration Gateway]
    Payroll[CTX-09 Payroll]
  end

  Ledger --> Identity
  Receivables --> Identity
  Receivables --> Ledger
  Payables --> Identity
  Payables --> Ledger
  Inventory --> Identity
  Banking --> Identity
  Banking --> Ledger
  Payments --> Identity
  Payments --> Receivables
  Projects --> Receivables
  Tax --> Identity
  Documents --> Identity
  Collab --> Receivables
  Collab --> Payables
  Reporting --> Ledger
  Reporting --> Receivables
  Reporting --> Payables
  Reporting --> Inventory
  Automation --> Receivables
  Automation --> Payables
  AI --> Reporting
  Integration --> Banking
  Integration --> Payments
  Integration --> Inventory
  Payroll --> Ledger

```

Payroll --> Identity

Rule of thumb enforced by this diagram: a module in Layer N may depend on any module in Layer 0..N-1, and may communicate with modules in the same layer only via domain events (never direct calls), to avoid layer-internal circular dependencies. Reporting (Layer 4) is read-only against every module it points to - it never writes back into Ledger, Receivables, Payables, or Inventory.

8. UML Domain Model (Class Diagrams)

These class diagrams define the canonical domain vocabulary that Volume 4 (Data Architecture) will turn into schema, and that Volume 9 (API Design) will turn into request/response payloads. Associations shown are domain relationships, not foreign keys - Volume 4 owns the physical implementation, including any denormalization for performance.

8.1 Ledger Core Domain

Diagram: UML class diagram - Ledger core domain

```
classDiagram
class Account {
+UUID id
+String code
+String name
+AccountType type
+UUID parentAccountId
+Boolean isSystemDefined
}
class JournalEntry {
+UUID id
+Date entryDate
+String reference
+JournalSource source
+Boolean isLocked
+postLines()
+reverse()
}
class JournalLine {
+UUID id
+UUID accountId
+Decimal debit
+Decimal credit
+String currency
+Decimal exchangeRate
}
class FiscalPeriod {
+Date startDate
+Date endDate
+Boolean isLocked
}
JournalEntry "1" *-- "2..*" JournalLine : contains
JournalLine "many" --> "1" Account : posts to
JournalEntry "many" --> "1" FiscalPeriod : belongs to
class AccountType {
<<enumeration>>
ASSET
LIABILITY
EQUITY
INCOME
EXPENSE
}
```

8.2 Receivables Core Domain

Diagram: UML class diagram - Receivables core domain

```

classDiagram
class Customer {
+UUID id
+String name
+String defaultCurrency
+PaymentTerms terms
}
class Quote {
+UUID id
+UUID customerId
+QuoteStatus status
+convertToSalesOrder()
}
class SalesOrder {
+UUID id
+UUID customerId
+SOSStatus status
+convertToInvoice()
}
class Invoice {
+UUID id
+UUID customerId
+InvoiceStatus status
+Decimal totalAmount
+Decimal balanceDue
+Date dueDate
+send()
+recordPayment()
+void()
}
class LineItem {
+UUID id
+UUID itemId
+Decimal quantity
+Decimal unitPrice
+Decimal taxAmount
}
class Payment {
+UUID id
+UUID invoiceId
+Decimal amount
+Date paymentDate
+PaymentMethod method
}
class CreditNote {
+UUID id
+UUID customerId
+Decimal amount
}
Customer "1" --> "many" Quote : requests
Customer "1" --> "many" Invoice : billed to
Quote "1" --> "0..1" SalesOrder : converts to
SalesOrder "1" --> "0..1" Invoice : converts to
Invoice "1" *-- "1..*" LineItem : contains
Invoice "1" --> "many" Payment : receives
Invoice "1" --> "0..many" CreditNote : offset by

```

9. Sequence Diagrams - Cross-Context Flows

These sequence diagrams elaborate the Vol. 1 Section 8 process flows at the module-interaction level, showing exactly which module calls which and where domain events are published versus consumed.

9.1 Invoice Creation & Ledger Posting

Diagram: Sequence - invoice send triggers ledger posting

```
sequenceDiagram
    participant UI as Web App
    participant GW as API Gateway
    participant AR as Receivables Module
    participant INV as Inventory Module
    participant TAX as Tax Module
    participant LED as Ledger Module
    participant BUS as Event Bus
    participant RPT as Reporting Module

    UI->>GW: POST /invoices (draft)
    GW->>AR: createInvoice(payload)
    AR->>INV: checkStock(lineItems)
    INV-->>AR: stock confirmed
    AR->>TAX: calculateTax(lineItems, customer)
    TAX-->>AR: tax breakdown
    AR->>AR: persist Invoice (status=Draft)
    UI->>GW: POST /invoices/{id}/send
    GW->>AR: sendInvoice(id)
    AR->>AR: update status=Sent
    AR->>BUS: publish InvoiceSent event
    BUS-->>LED: InvoiceSent
    LED->>LED: post JournalEntry (Dr AR, Cr Revenue)
    LED->>BUS: publish JournalPosted event
    BUS-->>RPT: JournalPosted
    RPT->>RPT: update read model / cached reports
    AR->>GW: 200 OK (invoice sent)
    GW-->>UI: 200 OK
```

9.2 Payment Received & Bank Reconciliation

Diagram: Sequence - payment matched during reconciliation

```
sequenceDiagram
    participant Bank as Integration Gateway (Bank Feed)
    participant BNK as Banking Module
    participant AR as Receivables Module
    participant LED as Ledger Module
    participant BUS as Event Bus

    Bank->>BNK: new bank transaction (deposit)
    BNK->>BNK: apply bank rules / suggest match
    BNK->>AR: query open invoices for customer
    AR-->>BNK: candidate invoice list
    BNK->>BNK: user confirms match (or auto-match)
    BNK->>AR: recordPayment(invoiceId, amount)
    AR->>AR: update invoice status (Paid/PartiallyPaid)
    AR->>BUS: publish PaymentRecorded event
    BUS-->>LED: PaymentRecorded
```

```

LED->>LED: post JournalEntry (Dr Bank, Cr AR)
BNK->>BNK: mark bank transaction Reconciled

```

9.3 Purchase 3-Way Match (Choreography)

Diagram: Sequence - event-driven 3-way match

```

sequenceDiagram
    participant AP as Payables Module
    participant INV as Inventory Module
    participant BUS as Event Bus
    participant LED as Ledger Module

    AP->>BUS: publish PurchaseOrderApproved
    BUS-->>INV: PurchaseOrderApproved (expect future receipt)
    INV->>BUS: publish GoodsReceived (when warehouse confirms)
    BUS-->>AP: GoodsReceived
    AP->>AP: vendor submits Bill (via portal or manual entry)
    AP->>AP: run 3-way match (PO vs Receipt vs Bill)
    alt match succeeds
        AP->>AP: auto-approve Bill
    else mismatch
        AP->>AP: flag for AP Clerk review (PER-05)
    end
    AP->>BUS: publish BillApproved
    BUS-->>LED: BillApproved
    LED->>LED: post JournalEntry (Dr Expense/Inventory, Cr AP)

```

10. Domain Event Catalog

The event bus (Section 5, 6) is the primary decoupling mechanism between bounded contexts at the same architectural layer. This table is the initial event contract registry; Volume 3 will define the concrete message schema/versioning strategy and Volume 9 will document the webhook-exposed subset for external integrators (BC-19).

Event	Published By	Consumed By	Purpose
InvoiceSent	Receivables	Ledger, Reporting, Collaboration	Trigger revenue posting and customer notification
PaymentRecorded	Receivables, Payables	Ledger, Banking, Reporting	Trigger cash posting and reconciliation matching
JournalPosted	Ledger	Reporting, AI Assistant, Automation	Update read models, trigger anomaly detection
BillApproved	Payables	Ledger, Reporting	Trigger AP posting
PurchaseOrderApproved	Payables	Inventory, Collaboration (vendor portal)	Notify vendor, prep for goods receipt
GoodsReceived	Inventory	Payables, Reporting	Enable 3-way match, update stock valuation
StockAdjusted	Inventory	Reporting, Ledger (if valuation impact)	Keep inventory valuation and COGS accurate
BankTransactionImported	Integration Gateway	Banking	Feed raw transactions for reconciliation
TenantProvisioned	Identity & Tenant	All modules	Initialize per-tenant defaults (COA template, roles)
RecurringInvoiceDue	Automation	Receivables	Trigger scheduled invoice generation
ApprovalRequested / ApprovalDecided	Receivables, Payables	Collaboration, Automation	Drive approval-workflow notifications
AnomalyDetected	AI Assistant	Reporting, Collaboration	Surface insight/alert to Business Owner

11. API Surface Overview (Context-Level)

Volume 9 will define full endpoint contracts; this section fixes which context owns which resource so Volume 9 does not have to re-derive ownership.

Bounded Context	Owns API Resources (examples)	Consumes From
Identity & Tenant	/orgs, /users, /roles, /auth	-
Ledger	/accounts, /journal-entries, /fiscal-periods	Identity
Receivables	/quotes, /sales-orders, /invoices, /credit-notes	Identity, Ledger, Inventory, Tax
Payables	/purchase-orders, /bills, /vendor-credits, /expenses	Identity, Ledger, Inventory, Tax
Banking	/bank-accounts, /bank-transactions, /reconciliations	Identity, Ledger
Payments	/payment-links, /payment-methods, /charges	Receivables, Integration Gateway
Inventory	/items, /price-lists, /stock-adjustments, /warehouses	Identity
Projects	/projects, /timesheets, /project-budgets	Receivables
Tax & Compliance	/tax-rates, /tax-exemptions, /1099-forms	Identity
Reporting & Analytics	/reports, /dashboards, /custom-reports	Ledger, Receivables, Payables, Inventory (read-only)
Documents	/documents, /attachments	Identity
Collaboration & Portals	/comments, /portal-sessions, /vendor-portal, /customer-portal	Receivables, Payables
Automation & Customization	/workflows, /custom-fields, /custom-templates	Receivables, Payables
AI Assistant	/assistant/query, /insights	Reporting
Integration Gateway	/integrations, /webhooks, /bank-feed-connections	Banking, Payments, Inventory

12. Traceability & Next Phase

12.1 Traceability to Volume 1

Vol. 2 Artifact	Traces To Vol. 1
Section 3.2 Bounded Context Catalog	Section 5 Business Capability Model (BC-01...BC-21)
Section 9 Sequence Diagrams	Section 7/8 Value Streams & Process Flows (VS-01...VS-06)
Section 2 Architectural Style Decision	Section 3 Business Architecture Principles (P1, P2)
Section 5 Container Diagram service split (Payroll, Payments, AI, Integration)	Section 6 Module Scope Catalog release phasing (R2/R3)

12.2 Forward Contract to Downstream Volumes

- Volume 3 (Technical Architecture) must adopt the 16 bounded contexts as its module/package structure and enforce the layering in Section 7 via lint rules or dependency-graph checks.
- Volume 4 (Data Architecture) must give each bounded context its own schema/namespace per Section 2's "no cross-module direct database access" rule, and use Section 8's class diagrams as the starting entity list.
- Volume 6 (Security Architecture) must derive its service-to-service auth model from the Section 5 container boundaries (which containers are network-reachable from which).
- Volume 9 (API Design) must expand Section 11's resource ownership table into full endpoint contracts, and Section 10's event catalog into the public webhook subset.

12.3 Open Questions for Sign-off

1. Is the modular-monolith-first decision (Section 2.1) acceptable given team size and timeline, or is there an organizational reason to start with separate deployables (e.g. separate teams per context)?
2. Should Payments (CTX-06) and Integration Gateway (CTX-16) be built as separate services from day one, given they inherently talk to volatile third-party APIs and benefit from independent deploy cycles even at MVP?
3. Confirm the event bus technology preference (if any exists) before Volume 3 - this affects whether Section 10's events are designed for at-least-once or exactly-once delivery semantics.

12.4 Next Phase

On approval, Phase 3 begins: Technical Architecture (Volume 3) - concrete technology stack selection with rationale, coding standards, folder/repository structure, logging strategy, caching strategy, and queue processing design, all built on the module boundaries fixed in this volume.